

Futexes!

The Good, the Bad and the Ugly!



Futexes!

The Good, the Bad and the Ugly!
(Part 1)

User space locks

- Let’s talk first about locks in user space

User space locks

- Let’s talk first about locks in user space
- It requires shared memory for the lock

User space locks

- Let’s talk first about locks in user space
- It requires shared memory for the lock
  - Could be from threads
  - Could be shared memory between processes

Spinning lock

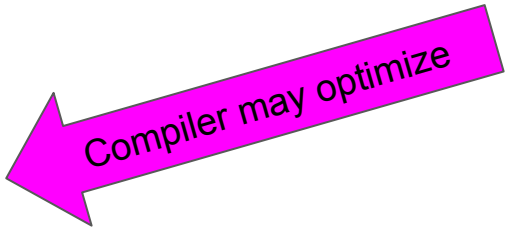
```
struct lock {
    long lock;
};

void lock(lock *l)
{
    while (l->lock)
        ;
    l->lock = 1;
}
```

Spinning lock

```
struct lock {
    long lock;
};

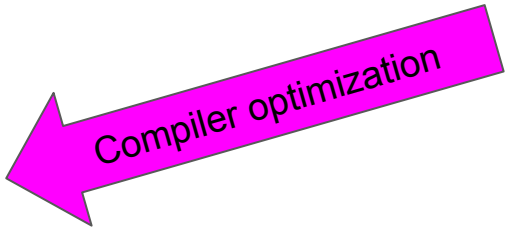
void lock(lock *l)
{
    while (l->lock)
        ;
    l->lock = 1;
}
```



Spinning lock

```
struct lock {
    long lock;
};

void lock(lock *l)
{
    if (l->lock) {
        for (;;)
            ;
    }
    l->lock = 1;
}
```



Spinning lock

```
struct lock {
    long lock;
};

void lock(lock *l)
{
    while (READ_ONCE(l->lock))
        ;
    l->lock = 1;
}
```

Spinning lock

```
struct lock {
    long lock;
};

void lock(lock *l)
{
    while (*(const volatile long *)&(l->lock))
        ;
    l->lock = 1;
}
```

Spinning lock

```
struct lock {
    long lock;
};

void lock(lock *l)
{
    while (l->lock)
        barrier();
    l->lock = 1;
}
```

Spinning lock

```
struct lock {
    long lock;
};

void lock(lock *l)
{
    while (l->lock)
        asm (“” ::: “memory”);
    l->lock = 1;
}
```

Spinning lock

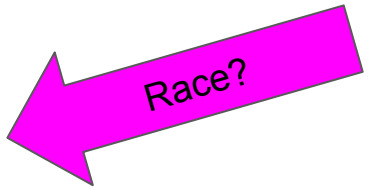
```
struct lock {
    long lock;
};

void lock(lock *l)
{
    while (l->lock)
        ;
    l->lock = 1;
}
```

Spinning lock

```
struct lock {
    long lock;
};

void lock(lock *l)
{
    while (l->lock)
        ;
    l->lock = 1;
}
```



Compare and Exchange

- Most architectures have a way to atomically update a variable

Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way

Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)

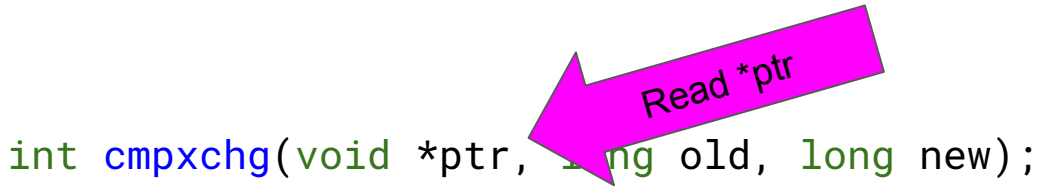
Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)

```
int cmpxchg(void *ptr, long old, long new);
```

Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)



Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)

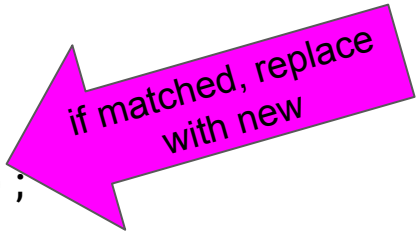



```
int cmpxchg(void *ptr, long old, long new);
```

Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)

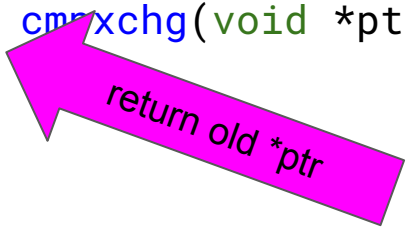
```
int cmpxchg(void *ptr, long old, long new);
```



Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)

```
int cmpxchg(void *ptr, long old, long new);
```



Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)

```
void lock(lock *l)
{
    while (l->lock)
        ;
    l->lock = 1;
}
```

Compare and Exchange

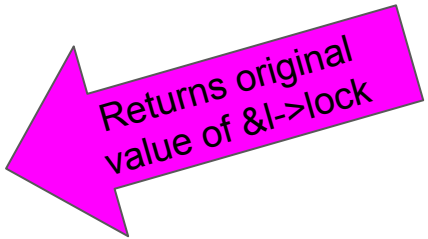
- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)

```
void lock(lock *l)
{
    int old;
    do {
        while (l->lock)
            ;
        old = cmpxchg(&l->lock, 0, 1);
    } while (old != 0);
}
```

Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)

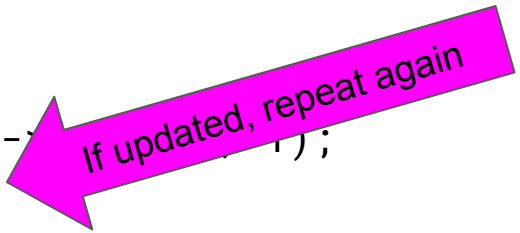
```
void lock(lock *l)
{
    int old;
    do {
        while (l->lock)
            ;
        old = cmpxchg(&l->lock, 0, 1);
    } while (old != 0);
}
```



Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)

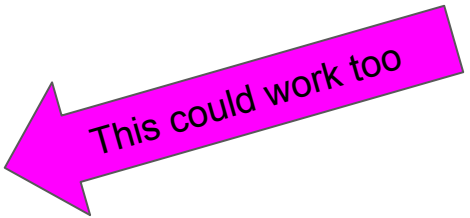
```
void lock(lock *l)
{
    int old;
    do {
        while (l->lock)
            ;
        old = cmpxchg(&l-
    } while (old != 0);
}
```



Compare and Exchange

- Most architectures have a way to atomically update a variable
- Compare and exchange is one such way
  - some architectures use Load Link and Store Conditional (ll/sc)

```
void lock(lock *l)
{
    int old;
    do {
        while (l->lock)
            ;
        old = xchg(&l->lock, 1);
    } while (old != 0);
}
```



Mutexes (Sleeping locks)

- Mutexes are safer when multiple tasks share the processor

Mutexes (Sleeping locks)

- Mutexes are safer when multiple tasks share the processor
- Blocked tasks will “sleep” and wait for the owner to wake it

Mutexes (Sleeping locks)

- Mutexes are safer when multiple tasks share the processor
- Blocked tasks will “sleep” and wait for the owner to wake it
- On Linux, a `futex` system call is used
  - Added in 2002, Linux version 2.5.6

Mutexes (Sleeping locks)

- Mutexes are safer when multiple tasks share the processor
- Blocked tasks will “sleep” and wait for the owner to wake it
- On Linux, a `futex` system call is used
  - Added in 2002, Linux version 2.5.6
- Avoids a system call in the non-contended case

The Good

futex - “Fast User space mutex”

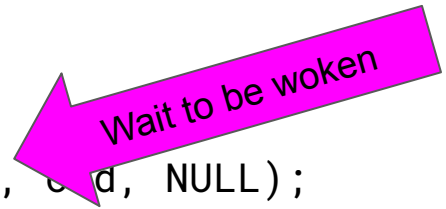
```
static unsigned int volatile lock_data;

void lock(unsigned int *lock)
{
    int old;
    do {
        old = cmpxchg(lock, 0, 1);
        if (old)
            futex(lock, FUTEX_WAIT, old, NULL);
    } while (old != 0);
}
```

futex - “Fast User space mutex”

```
static unsigned int volatile lock_data;

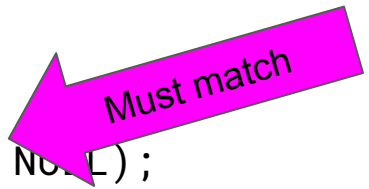
void lock(unsigned int *lock)
{
    int old;
    do {
        old = cmpxchg(lock, 0, 1);
        if (old)
            futex(lock, FUTEX_WAIT, old, NULL);
    } while (old != 0);
}
```



futex - “Fast User space mutex”

```
static unsigned int volatile lock_data;

void lock(unsigned int *lock)
{
    int old;
    do {
        old = cmpxchg(lock, 0, 1);
        if (old)
            futex(lock, FUTEX_WAIT, old, NULL);
    } while (old != 0);
}
```



futex - “Fast User space mutex”

```
void unlock(unsigned int *lock)
{
    *lock = 0;
    mb();
    futex(lock, FUTEX_WAKE, 1);
}
```

futex - “Fast User space mutex”

```
void unlock(unsigned int *lock)
{
    *lock = 0;
    mb();
    futex(lock, FUTEX_WAKE, 1);
}
```



futex - “Fast User space mutex”

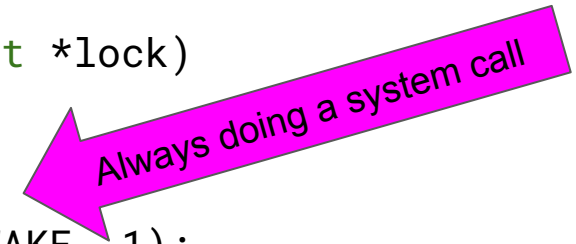
```
void unlock(unsigned int *lock)
{
    *lock = 0;
    mb();
    futex(lock, FUTEX_WAKE, 1);
}
```



futex - “Fast User space mutex”

Naive implementation

```
void unlock(unsigned int *lock)
{
    *lock = 0;
    mb();
    futex(lock, FUTEX_WAKE, 1);
}
```



```
void lock(unsigned int *lock)
{
    int old2, old = 0, new = 1;
    for (;;) {
        old2 = old;
        old = cmpxchg(lock, old2, new);
        if (old == old2)
            return;
check:
        switch (old) {
        case 0: new = 1; continue;
        case 2: new = 3; continue;
        }
        old2 = old;
        old = cmpxchg(lock, old2, 3);
        if (old != old2) goto check;
        futex(lock, FUTEX_WAIT, 3, NULL);
        old = *lock & ~1;
        goto check;
    }
}
```

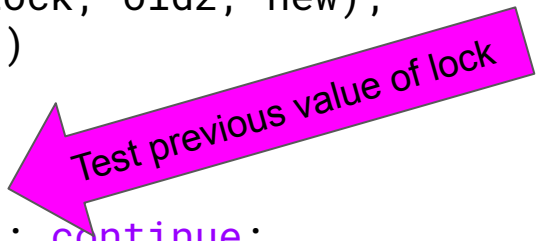
```
void lock(unsigned int *lock)
{
    int old2, old = 0, new = 1;
    for (;;) {
        old2 = old;
        old = cmpxchg(lock, old2, new);
        if (old == old2)
            return;
check:
        switch (old) {
        case 0: new = 1; continue;
        case 2: new = 3; continue;
        }
        old2 = old;
        old = cmpxchg(lock, old2, 3);
        if (old != old2) goto check;
        futex(lock, FUTEX_WAIT, 3, NULL);
        old = *lock & ~1;
        goto check;
    }
}
```



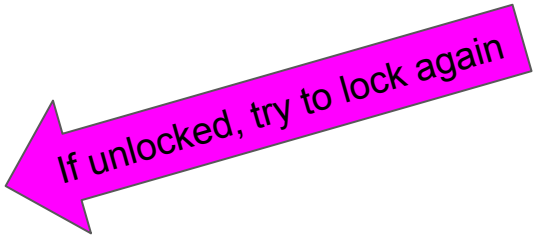
```
void lock(unsigned int *lock)
{
    int old2, old = 0, new = 1;
    for (;;) {
        old2 = old;
        old = cmpxchg(lock, old2
        if (old == old2)
            return;
check:
        switch (old) {
        case 0: new = 1; continue;
        case 2: new = 3; continue;
        }
        old2 = old;
        old = cmpxchg(lock, old2, 3);
        if (old != old2) goto check;
        futex(lock, FUTEX_WAIT, 3, NULL);
        old = *lock & ~1;
        goto check;
    }
}
```



```
void lock(unsigned int *lock)
{
    int old2, old = 0, new = 1;
    for (;;) {
        old2 = old;
        old = cmpxchg(lock, old2, new);
        if (old == old2)
            return;
check:
        switch (old) {
        case 0: new = 1; continue;
        case 2: new = 3; continue;
        }
        old2 = old;
        old = cmpxchg(lock, old2, 3);
        if (old != old2) goto check;
        futex(lock, FUTEX_WAIT, 3, NULL);
        old = *lock & ~1;
        goto check;
    }
}
```



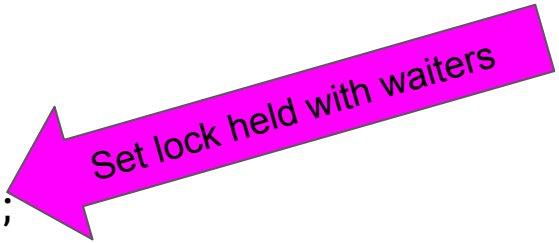
```
void lock(unsigned int *lock)
{
    int old2, old = 0, new = 1;
    for (;;) {
        old2 = old;
        old = cmpxchg(lock, old2, new);
        if (old == old2)
            return;
check:
        switch (old) {
        case 0: new = 1; continue;
        case 2: new = 3; continue;
        }
        old2 = old;
        old = cmpxchg(lock, old2, 3);
        if (old != old2) goto check;
        futex(lock, FUTEX_WAIT, 3, NULL);
        old = *lock & ~1;
        goto check;
    }
}
```



```
void lock(unsigned int *lock)
{
    int old2, old = 0, new = 1;
    for (;;) {
        old2 = old;
        old = cmpxchg(lock, old2, new);
        if (old == old2)
            return;
check:
        switch (old) {
        case 0: new = 1; continue;
        case 2: new = 3; continue;
        }
        old2 = old;
        old = cmpxchg(lock, old2, 3);
        if (old != old2) goto check;
        futex(lock, FUTEX_WAIT, 3, NULL);
        old = *lock & ~1;
        goto check;
    }
}
```



```
void lock(unsigned int *lock)
{
    int old2, old = 0, new = 1;
    for (;;) {
        old2 = old;
        old = cmpxchg(lock, old2, new);
        if (old == old2)
            return;
check:
        switch (old) {
        case 0: new = 1; continue;
        case 2: new = 3; continue;
        }
        old2 = old;
        old = cmpxchg(lock, old2, 3);
        if (old != old2) goto check;
        futex(lock, FUTEX_WAIT, 3, NULL);
        old = *lock & ~1;
        goto check;
    }
}
```



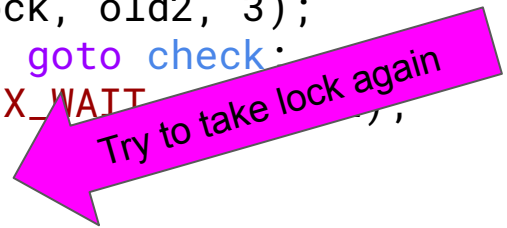
```
void lock(unsigned int *lock)
{
    int old2, old = 0, new = 1;
    for (;;) {
        old2 = old;
        old = cmpxchg(lock, old2, new);
        if (old == old2)
            return;
check:
        switch (old) {
        case 0: new = 1; continue;
        case 2: new = 3; continue;
        }
        old2 = old;
        old = cmpxchg(lock, old2, 3);
        if (old != old2) goto check;
        futex(lock, FUTEX_WAIT, 3, NULL);
        old = *lock & ~1;
        goto check;
    }
}
```



```
void lock(unsigned int *lock)
{
    int old2, old = 0, new = 1;
    for (;;) {
        old2 = old;
        old = cmpxchg(lock, old2, new);
        if (old == old2)
            return;
check:
        switch (old) {
        case 0: new = 1; continue;
        case 2: new = 3; continue;
        }
        old2 = old;
        old = cmpxchg(lock, old2, 3);
        if (old != old2) goto check;
        futex(lock, FUTEX_WAIT, 3, NULL);
        old = *lock & ~1;
        goto check;
    }
}
```



```
void lock(unsigned int *lock)
{
    int old2, old = 0, new = 1;
    for (;;) {
        old2 = old;
        old = cmpxchg(lock, old2, new);
        if (old == old2)
            return;
check:
        switch (old) {
        case 0: new = 1; continue;
        case 2: new = 3; continue;
        }
        old2 = old;
        old = cmpxchg(lock, old2, 3);
        if (old != old2) goto check;
        futex(lock, FUTEX_WAIT
        old = *lock & ~1;
        goto check;
    }
}
```

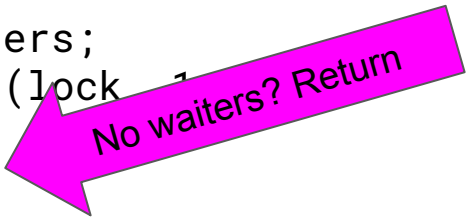


```
void unlock(unsigned int *lock)
{
    int old, waiters;
    old = cmpxchg(lock, 1, 0);
    if (old == 1)
        return;
    *lock = 2;
    waiters = futex(lock, FUTEX_WAKE, 1);
    if (!waiters)
        cmpxchg(lock, 2, 0);
}
```

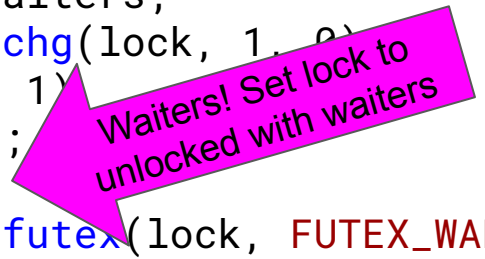
```
void unlock(unsigned int *lock)
{
    int old, waiters;
    old = cmpxchg(lock, 1, 0);
    if (old == 1)
        return;
    *lock = 2;
    waiters = futex(lock, FUTEX_WAKE, 1);
    if (!waiters)
        cmpxchg(lock, 2, 0);
}
```



```
void unlock(unsigned int *lock)
{
    int old, waiters;
    old = cmpxchg(lock, 1
    if (old == 1)
        return;
    *lock = 2;
    waiters = futex(lock, FUTEX_WAKE, 1);
    if (!waiters)
        cmpxchg(lock, 2, 0);
}
```



```
void unlock(unsigned int *lock)
{
    int old, waiters;
    old = cmpxchg(lock, 1, 0)
    if (old == 1)
        return;
    *lock = 2;
    waiters = futex(lock, FUTEX_WAKE, 1);
    if (!waiters)
        cmpxchg(lock, 2, 0);
}
```



```
void unlock(unsigned int *lock)
{
    int old, waiters;
    old = cmpxchg(lock, 1, 0);
    if (old == 1)
        return;
    *lock = 2;
    waiters = futex(lock, FUTEX_WAKE, 1);
    if (!waiters)
        cmpxchg(lock, 2, 0);
}
```



```
void unlock(unsigned int *lock)
{
    int old, waiters;
    old = cmpxchg(lock, 1, 0);
    if (old == 1)
        return;
    *lock = 2;
    waiters = futex                    FUTEX_WAKE, 1);
    if (!waiters)
        cmpxchg(lock, 2, 0);
}
```

No waiters?

```
void unlock(unsigned int *lock)
{
    int old, waiters;
    old = cmpxchg(lock, 1, 0);
    if (old == 1)
        return;
    *lock = 2;
    waiters = futex(lock, FUTEX_WAKE
    if (!waiters)
        cmpxchg(lock, 2, 0);
}
```



```
void unlock(unsigned int *lock)
{
    int old, waiters;
    old = cmpxchg(lock, 1, 0);
    if (old == 1)
        return;
    *lock = 2;
    waiters = futex(lock, FUTEX_WAKE, 1);
    if (!waiters)
        cmpxchg(lock, 2, 0);
```



Futex system call

- Allows user space to synchronize with other threads or processes

Futex system call

- Allows user space to synchronize with other threads or processes
- Threads are easy as they share the same memory space





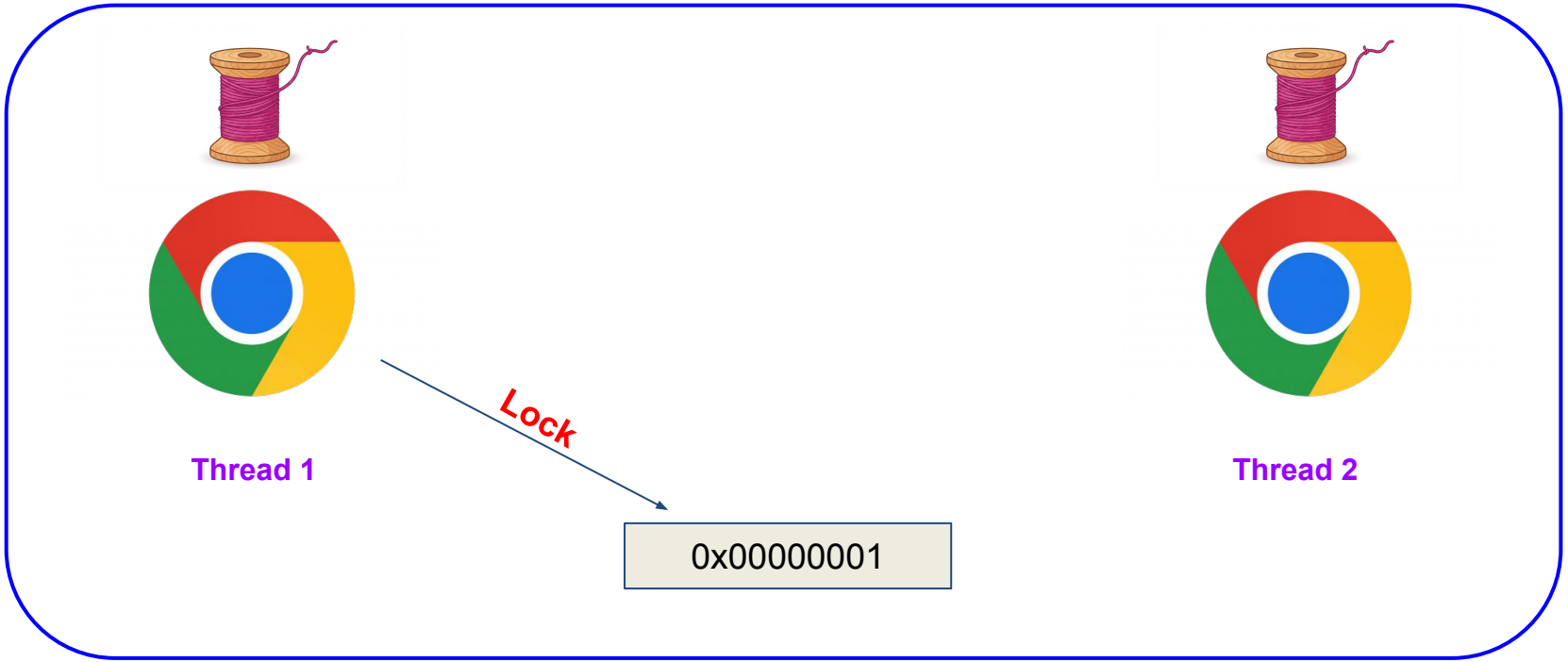
Thread 1

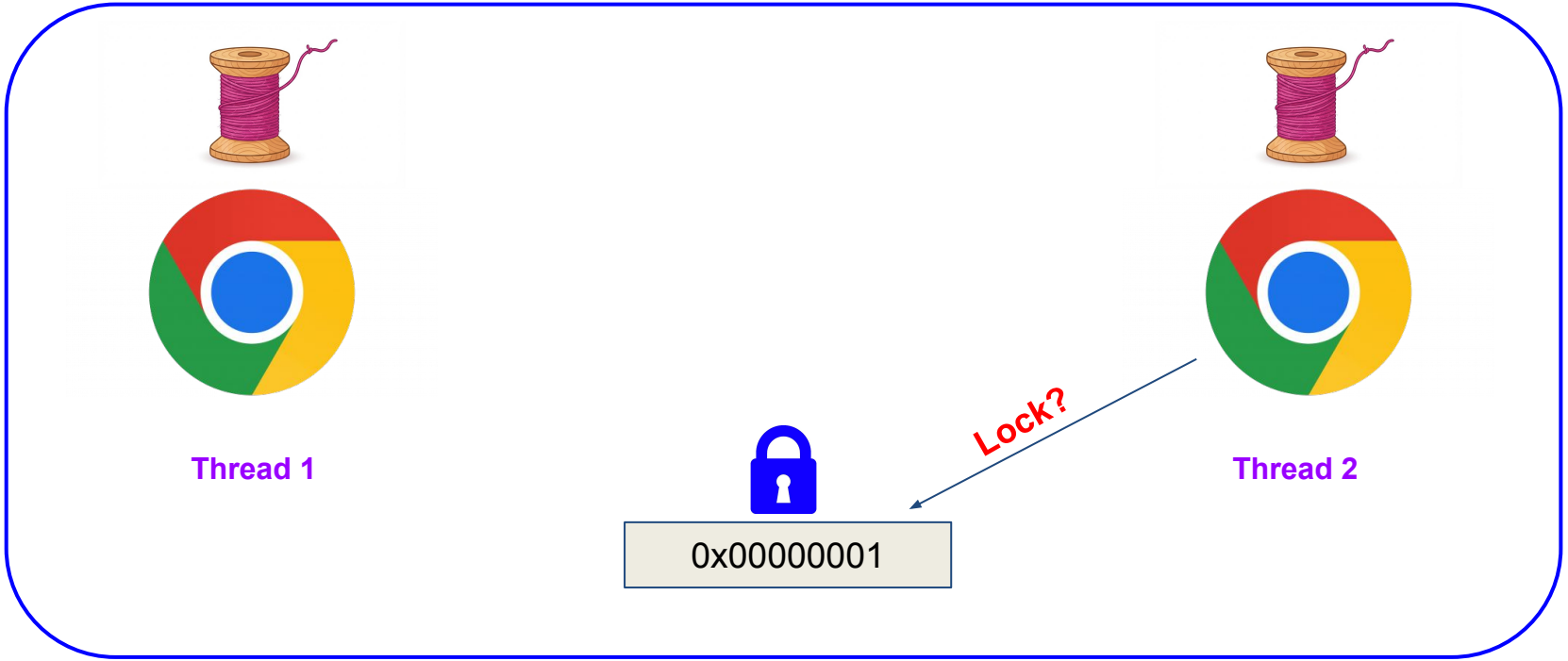


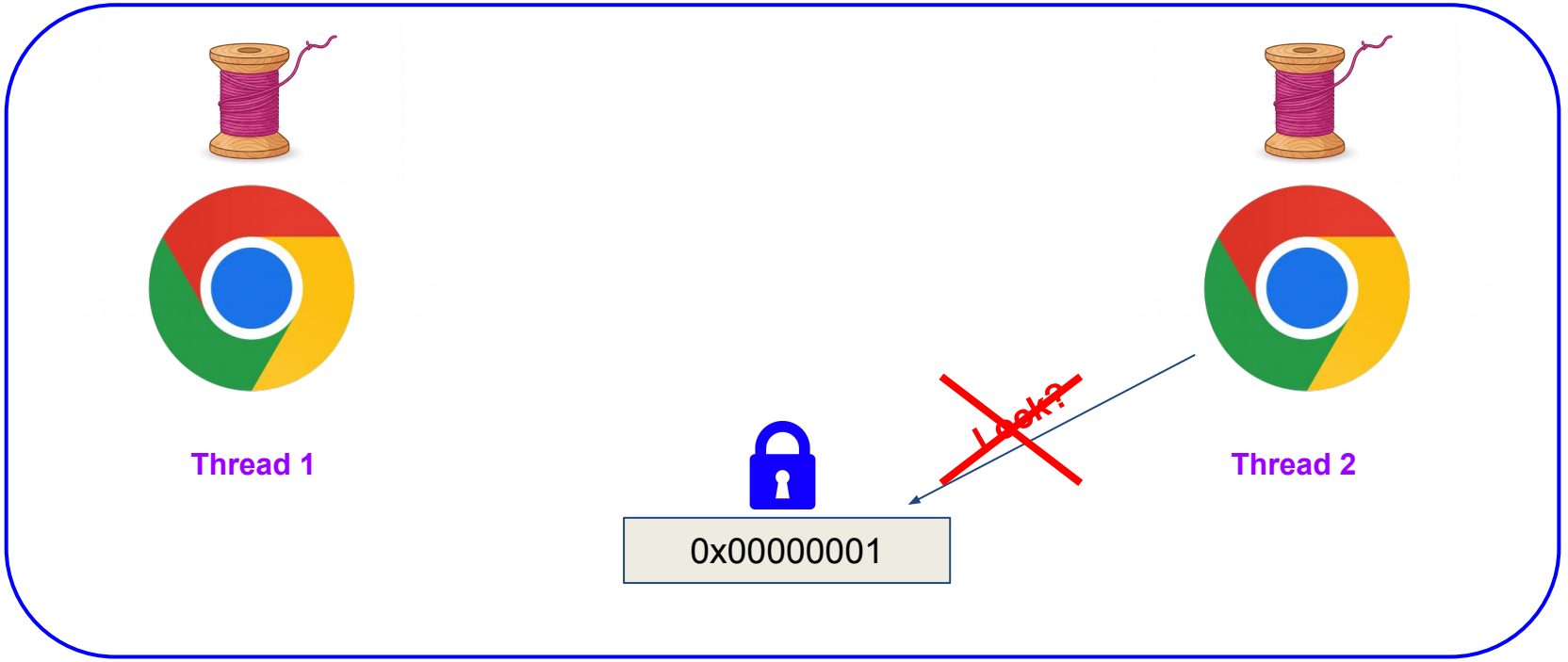


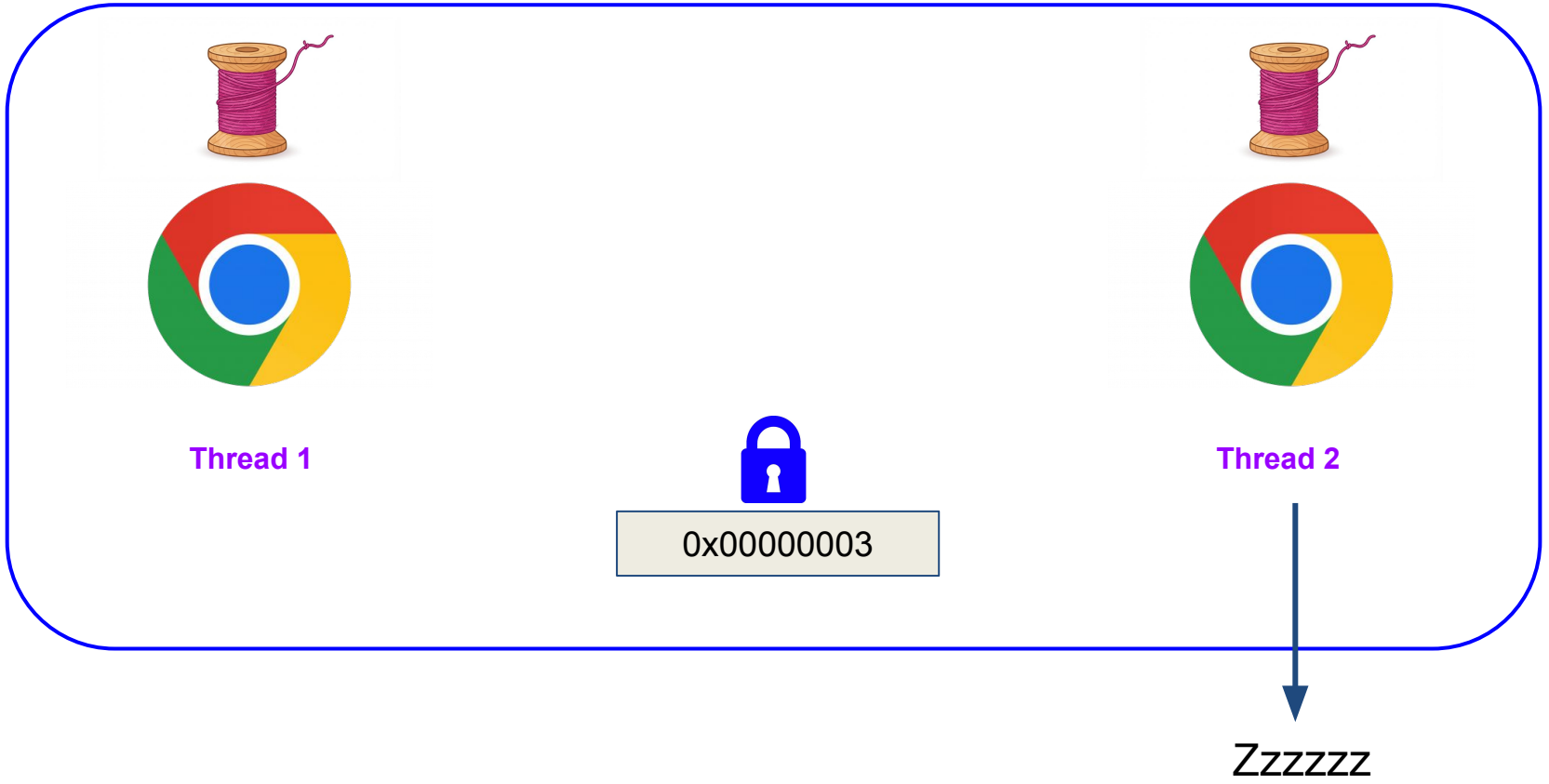
Thread 2

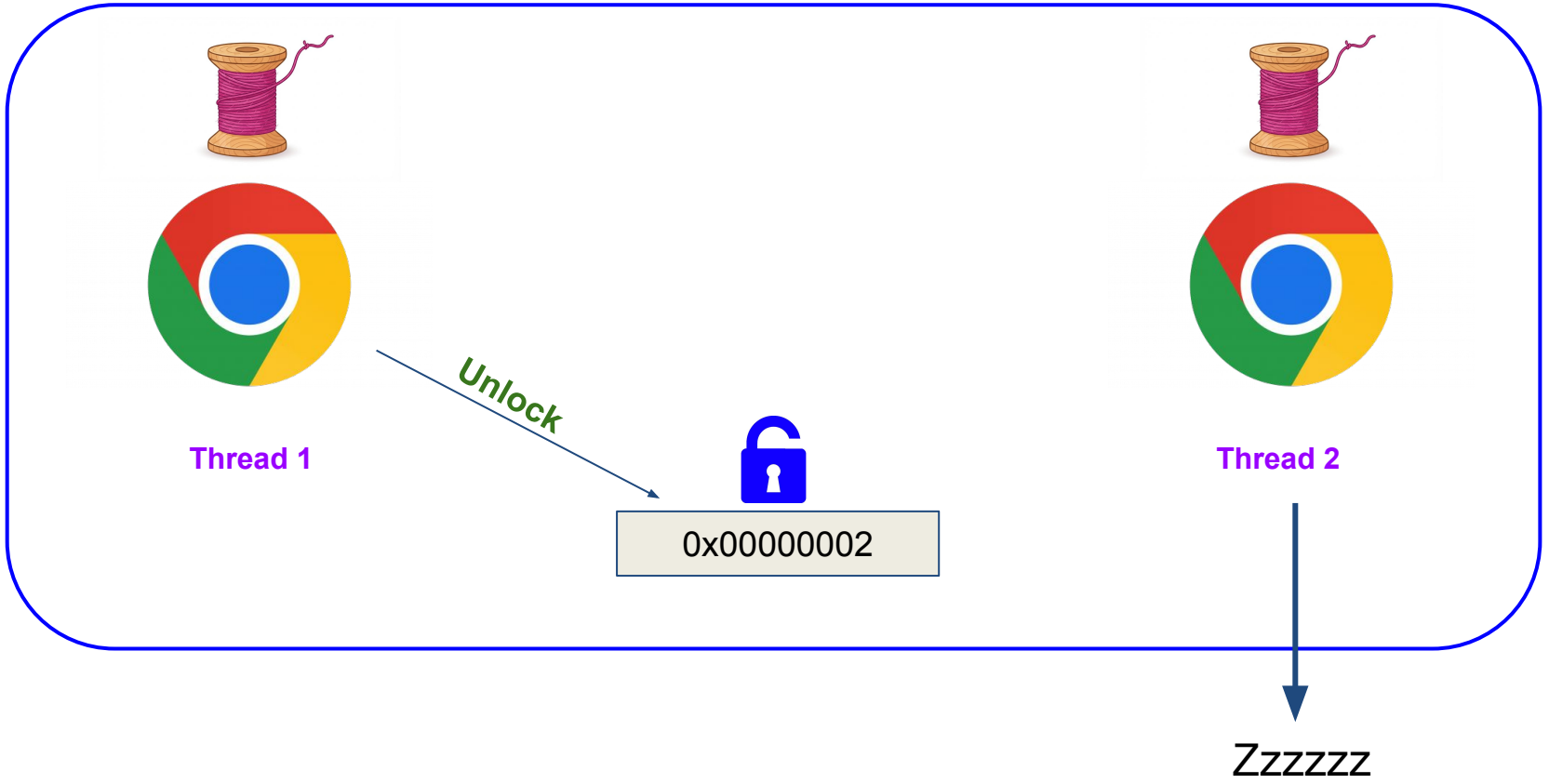
0x00000000

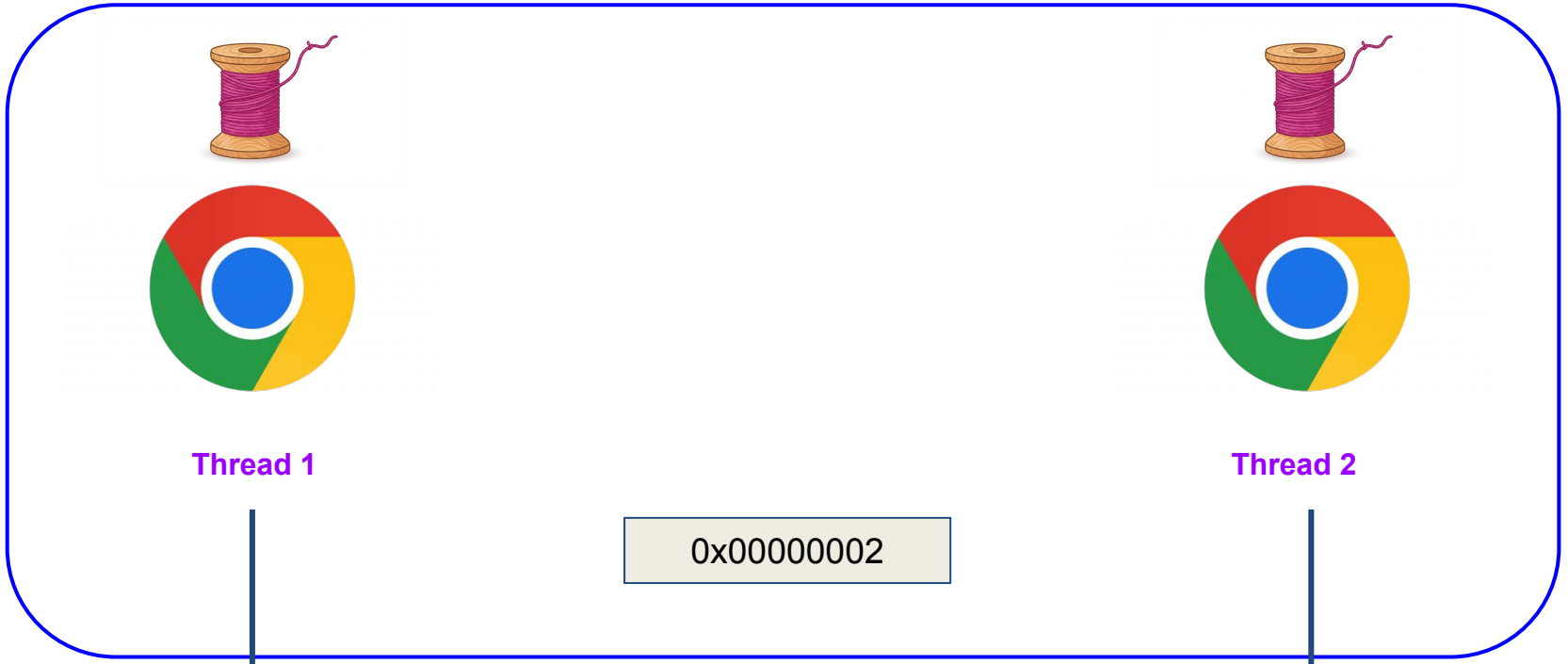

Lock
Thread 1
Thread 2
0x00000001


Thread 1
Lock?
Thread 2
0x00000001

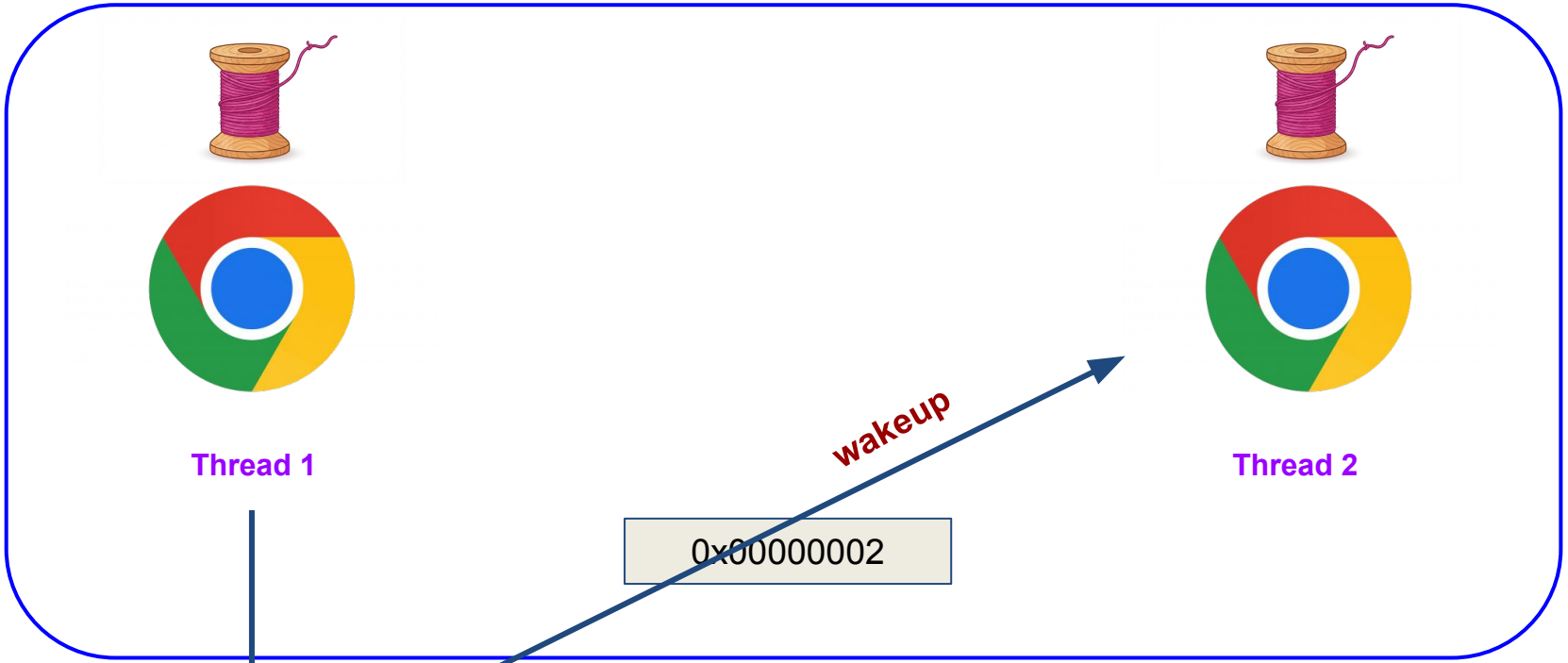

Lock?
Thread 1
Thread 2
0x00000001


Thread 1
0x00000003
Thread 2
Zzzzzz

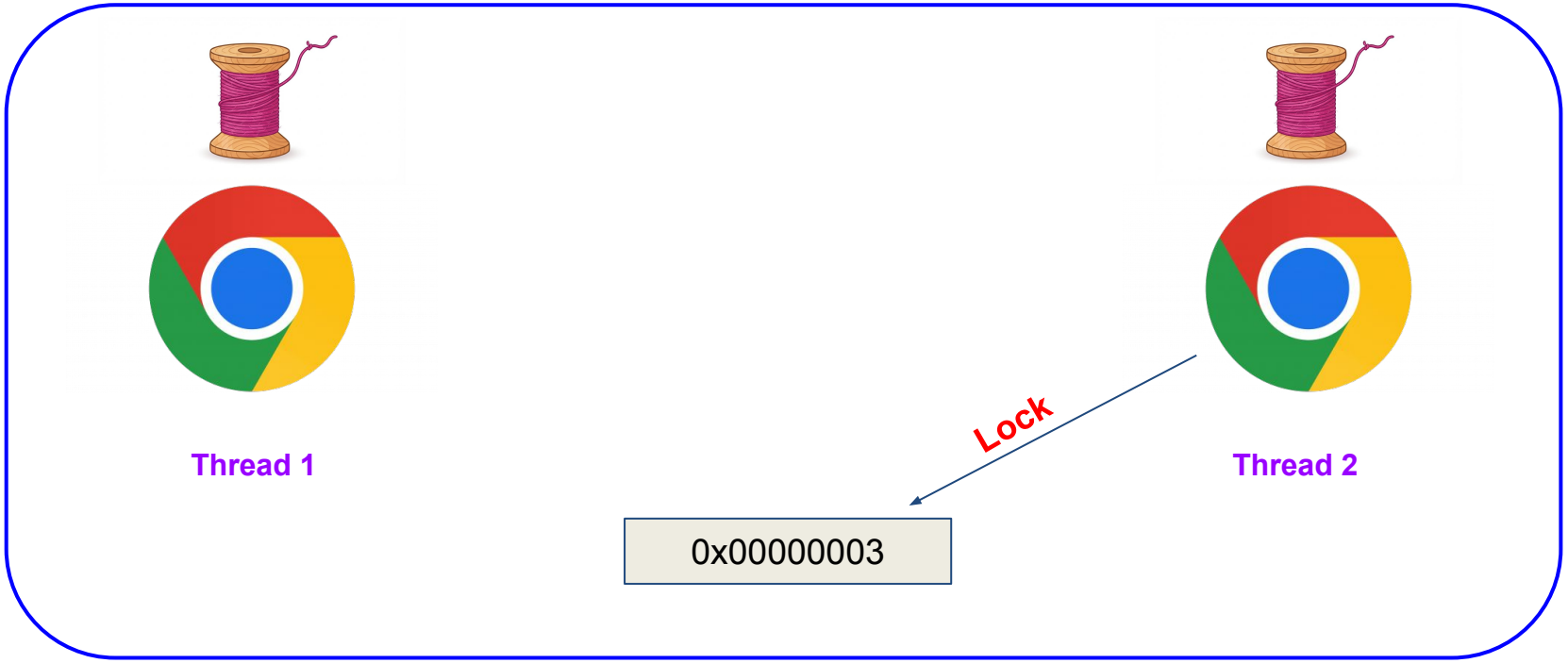

Unlock
Thread 1
Thread 2
0x00000002
Zzzzzz




```
futex(lock, FUTEX_WAKE, 1);
```




```
futex(lock, FUTEX_WAKE, 1);
```


Thread 1
Lock
Thread 2
0x00000003

Futex system call

- Processes need to add shared memory (mmap or shmem)

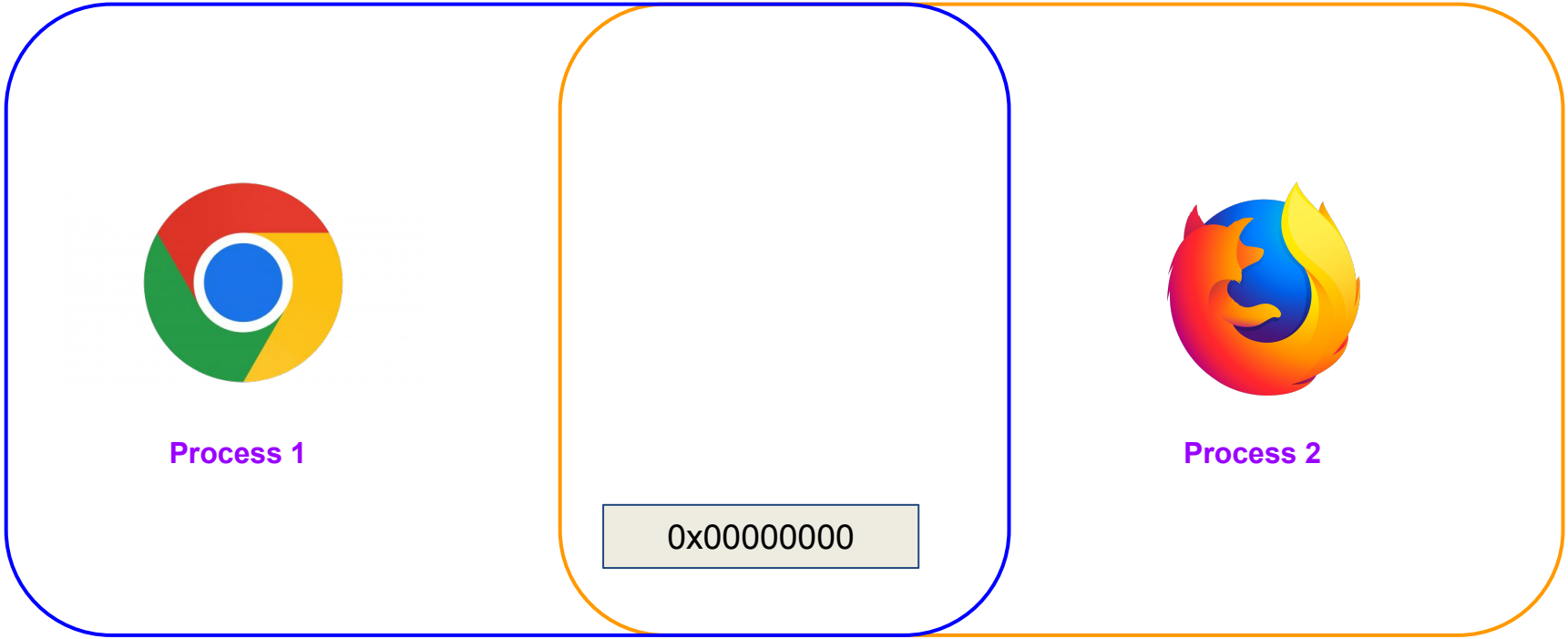
Futex system call

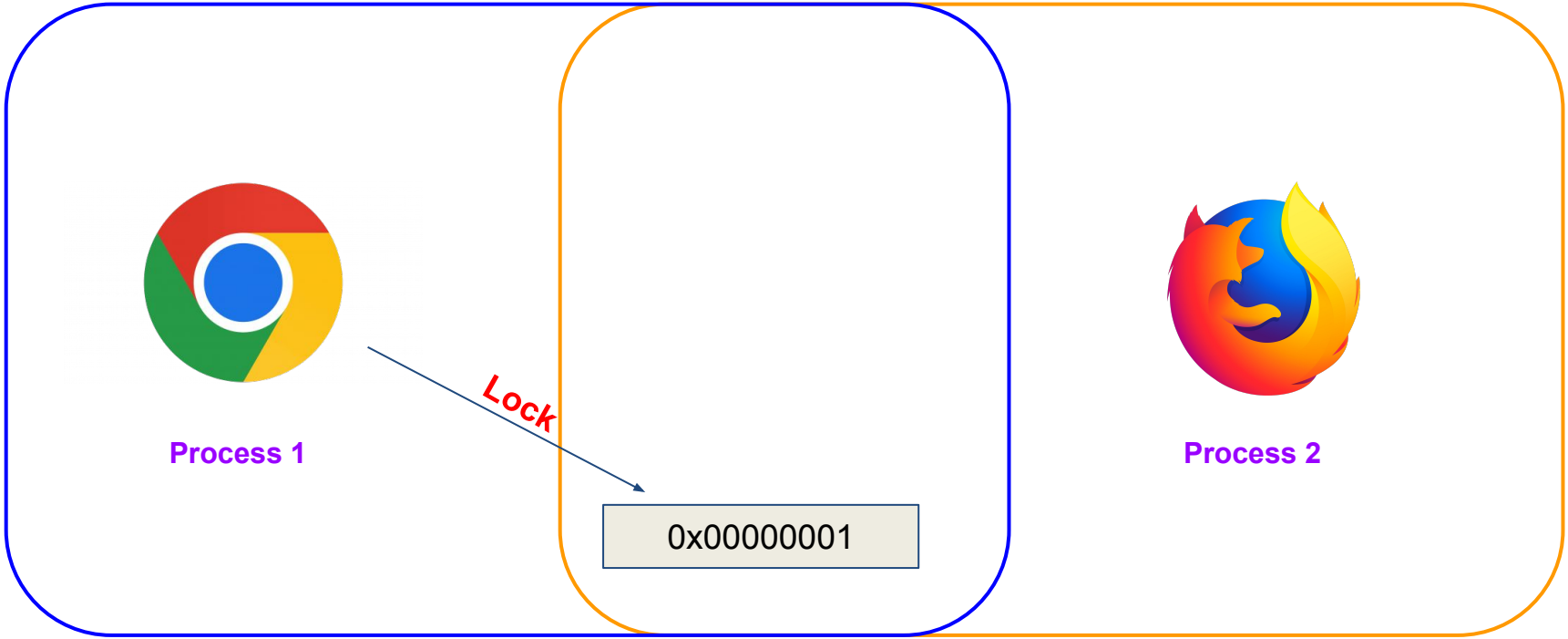
- Processes need to add shared memory (mmap or shmem)
- This usually means that the memory is not associated with a single task

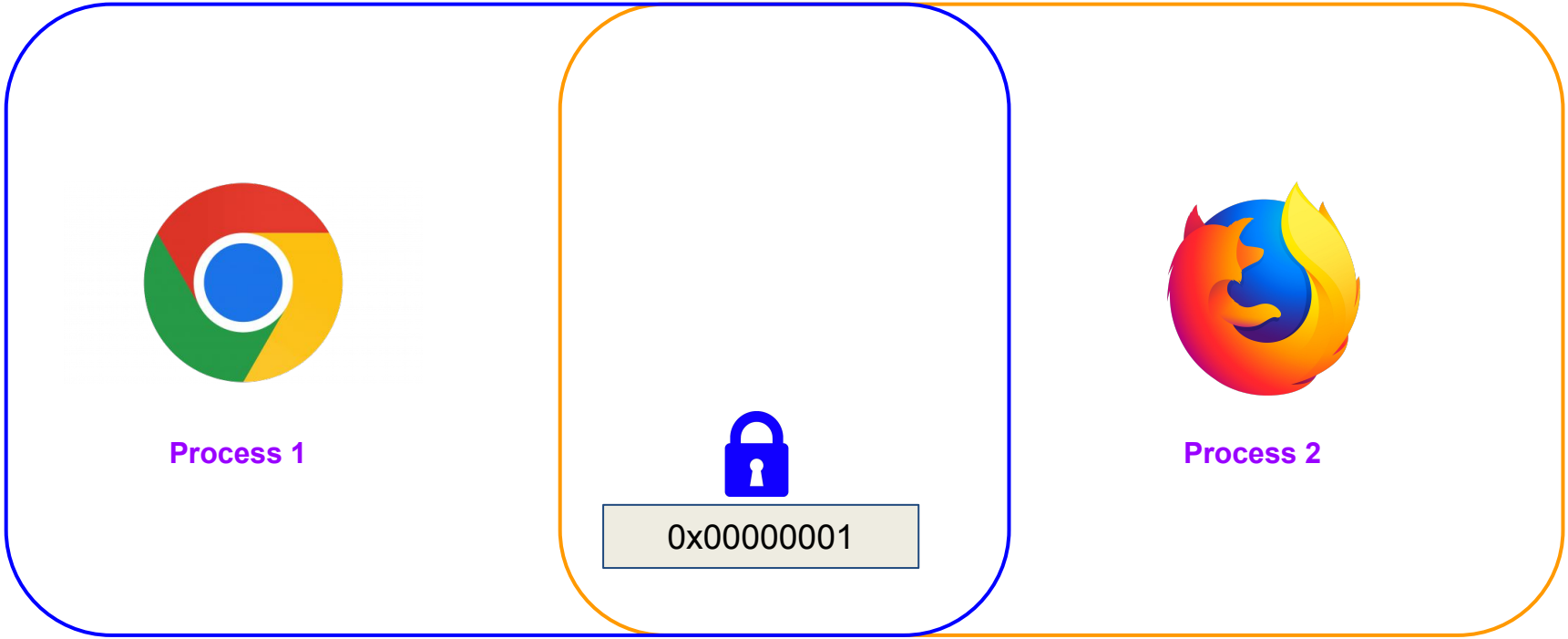
Futex system call

- Processes need to add shared memory (mmap or shmem)
- This usually means that the memory is not associated with a single task
- If that task exits, the memory is still persistent

The Bad


Process 1
0x00000000
Process 2


Process 1
Lock
0x00000001
Process 2


Process 1
0x00000001
Process 2



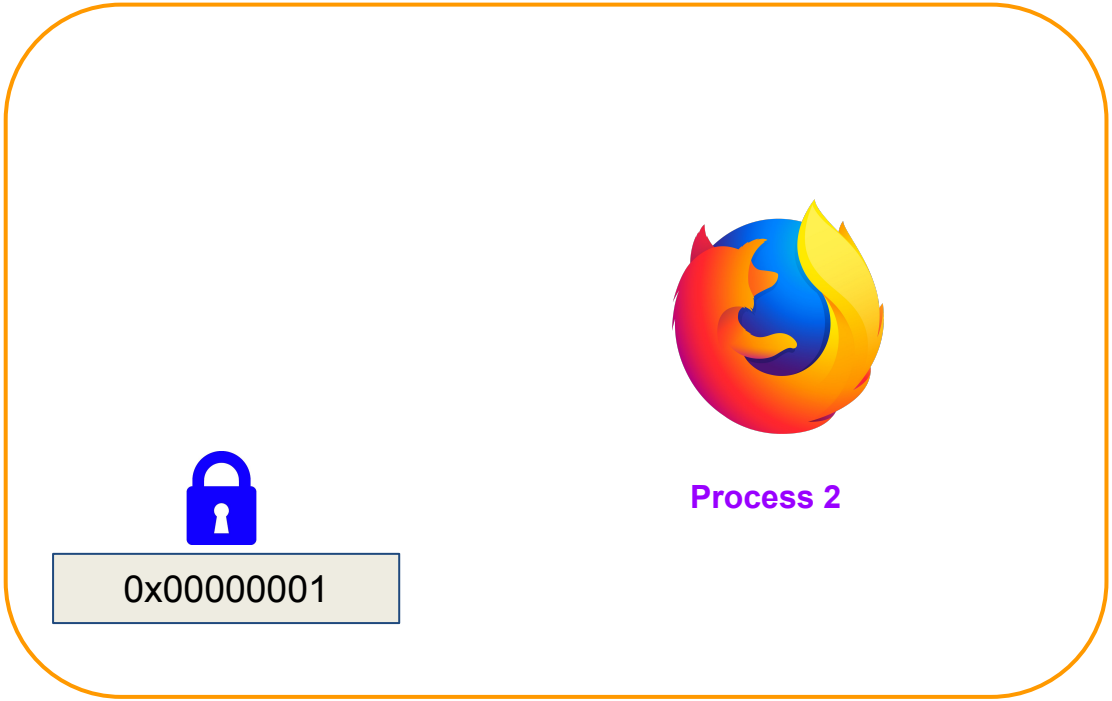
Process 1

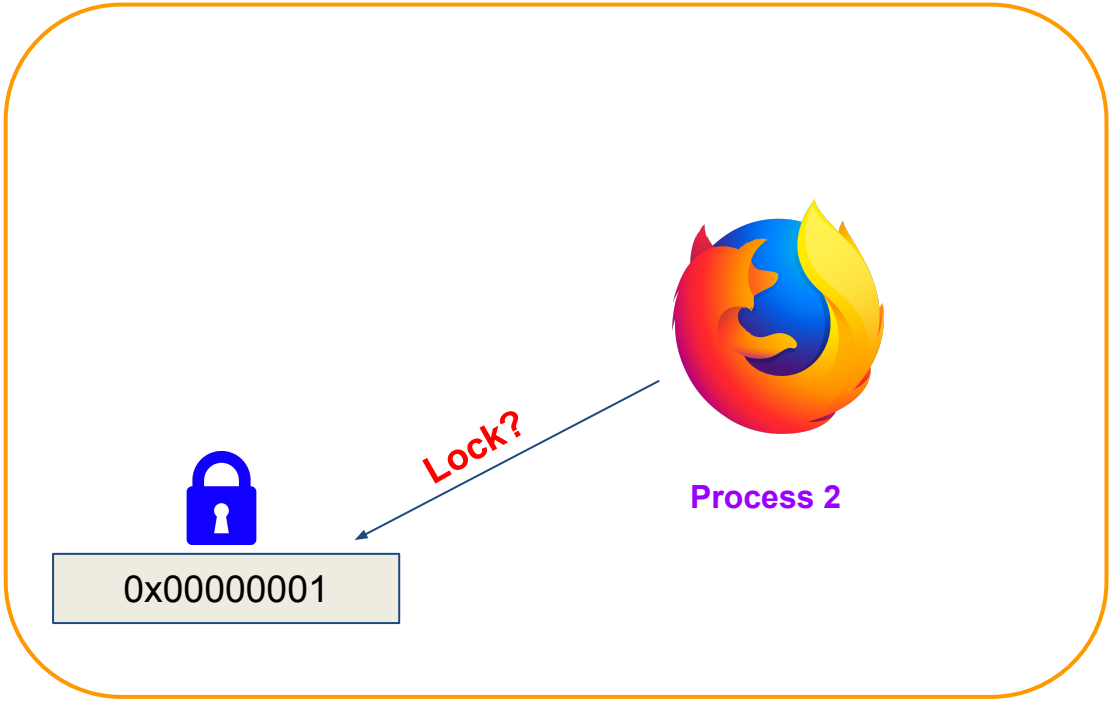


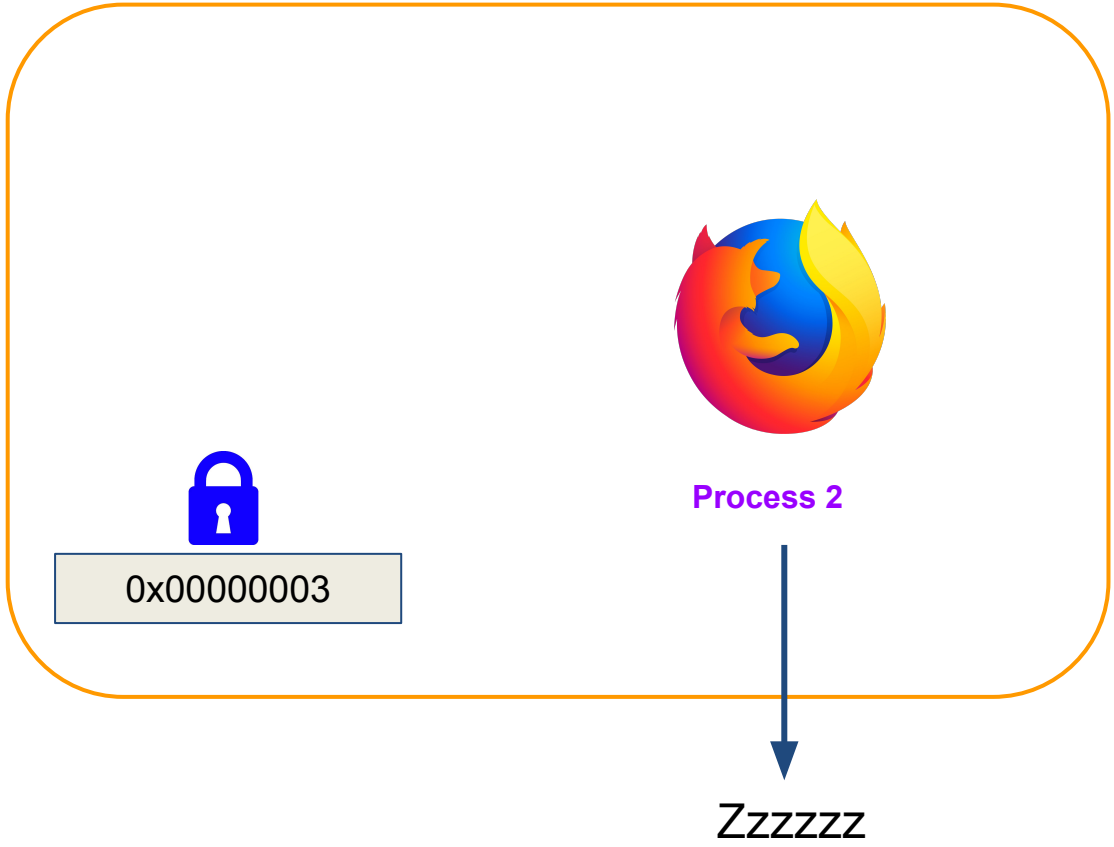
0x00000001



Process 2


Process 2
0x00000001


Lock?
Process 2
0x00000001


Process 2
0x00000003
Zzzzzz





0x00000001



Process 2

Zzzzzz

Robust Futexes

- Robust futexes solve the problem of an owner crashing

Robust Futexes

- Robust futexes solve the problem of an owner crashing
- It has its own API to define a “contract” between user space and the kernel

Robust Futexes

- Robust futexes solve the problem of an owner crashing
- It has its own API to define a “contract” between user space and the kernel
- A per-thread list of locks owned by the thread

Robust Futexes

- Robust futexes solve the problem of an owner crashing
- It has its own API to define a “contract” between user space and the kernel
- A per-thread list of locks owned by the thread
  - New system call to tell kernel where the head of the list is
  - `set_robust_list(head, len);`

Robust Futexes

- Robust futexes solve the problem of an owner crashing
- It has its own API to define a “contract” between user space and the kernel
- A per-thread list of locks owned by the thread
  - New system call to tell kernel where the head of the list is
  - `set_robust_list(head, len);`
- The mutex holds the pointer to the next lock

Robust Futexes

- Robust futexes solve the problem of an owner crashing
- It has its own API to define a “contract” between user space and the kernel
- A per-thread list of locks owned by the thread
  - New system call to tell kernel where the head of the list is
  - `set_robust_list(head, len);`
- The mutex holds the pointer to the next lock
- Only the owner can set and clear that link list

```c
struct robust_list {
     struct robust_list  *next;
};

struct robust_list_head {
     struct robust_list  list;
     long                offset;
     struct robust_list  *list_op_pending;
};

struct my_mutex {
     unsigned int        lock;
     struct robust_list  list;
};

#define FUTEX_WAITERS         0x80000000
#define FUTEX_OWNER_DIED      0x40000000

void init_thread(mythread_t *thread)
{
     struct robust_list_head *rl = &thread->robust_list;
     memset(rl, 0, sizeof(*rl));
     rl->list.next = &rl->list;
     rl->list.offest = offsetof(struct my_mutex, lock) -
                       offsetof(struct my_mutex, list);
     set_robust_list(rb, sizeof(*rl));
}
```

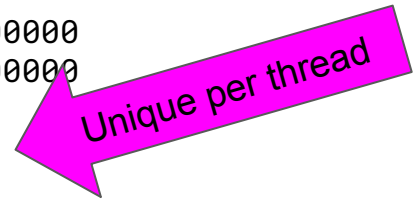
```c
struct robust_list {
     struct robust_list  *next;
};

struct robust_list_head {
     struct robust_list  list;
     long                offset;
     struct robust_list  *list_op_pending;
};

struct my_mutex {
     unsigned int        lock;
     struct robust_list  list;
};

#define FUTEX_WAITERS         0x80000000
#define FUTEX_OWNER_DIED      0x40000000

void init_thread(mythread_t *thread)
{
     struct robust_list_head *rl = &thread->robust_list;
     memset(rl, 0, sizeof(*rl));
     rl->list.next = &rl->list;
     rl->list.offest = offsetof(struct my_mutex, lock) -
                       offsetof(struct my_mutex, list);
     set_robust_list(rb, sizeof(*rl));
}
```



```c
struct robust_list {
     struct robust_list  *next;
};

struct robust_list_head {
     struct robust_list  list;
     long                offset;
     struct robust_list  *list_op_pending;
};

struct my_mutex {
     unsigned int        lock;
     struct robust_list  list;
};

#define FUTEX_WAITERS         0x80000000
#define FUTEX_OWNER_DIED      0x40000000

void init_thread(mythread_t *thread)
{
     struct robust_list_head *rl = &thread->robust_list;
     memset(rl, 0, sizeof(*rl));
     rl->list.next = &rl->list;
     rl->list.offest = offsetof(struct my_mutex, lock) -
                       offsetof(struct my_mutex, list);
     set_robust_list(rb, sizeof(*rl));
}
```



```c
struct robust_list {
     struct robust_list  *next;
};

struct robust_list_head {
     struct robust_list  list;
     long                offset;
     struct robust_list  *list_op_pending;
};

struct my_mutex {
     unsigned int        lock;
     struct robust_list  list;
};

#define FUTEX_WAITERS         0x80000000
#define FUTEX_OWNER_DIED      0x40000000

void init_thread(mythread_t *thread)
{
     struct robust_list_head *rl = &th robust_list;
     memset(rl, 0, sizeof(*rl));
     rl->list.next = &rl->list;
     rl->list.offest = offsetof(struct my_mutex, lock) -
                       offsetof(struct my_mutex, list);
     set_robust_list(rb, sizeof(*rl));
}
```



```c
struct robust_list {
     struct robust_list  *next;
};

struct robust_list_head {
     struct robust_list  list;
     long                offset;
     struct robust_list  *list_op_pending;
};

struct my_mutex {
     unsigned int        lock;
     struct robust_list  list;
};

#define FUTEX_WAITERS         0x80000000
#define FUTEX_OWNER_DIED      0x40000000

void init_thread(mythread_t *thread)
{
     struct robust_list_head *rl = &thread->robust_list;
     memset(rl, 0, sizeof(*rl));
     rl->list.next = &rl->list;
     rl->list.offest = offsetof(struct my_mutex, lock)
                       offsetof(struct my_mutex, list);
     set_robust_list(rb, sizeof(*rl));
}
```



```c
struct robust_list {
     struct robust_list  *next;
};

struct robust_list_head {
     struct robust_list  list;
     long                offset;
     struct robust_list  *list_op_pending;
};

struct my_mutex {
     unsigned int        lock;
     struct robust_list  list;
};

#define FUTEX_WAITERS         0x80000000
#define FUTEX_OWNER_DIED      0x40000000

void init_thread(mythread_t *thread)
{
     struct robust_list_head *rl = &thread->robust_list;
     memset(rl, 0, sizeof(*rl));
     rl->list.next = &rl->list;
     rl->list.offest = offsetof(struct my_mut
                       offsetof(struct m          ist);
     set_robust_list(rb, sizeof(*rl));
}
```



```
struct robust_list {
     struct robust_list  *next;
};

struct robust_list_head {
     struct robust_list  list;
     long                offset;
     struct robust_list  *list_op_pending;
};

struct my_mutex {
     unsigned int        lock;
     struct robust_list  list;
};

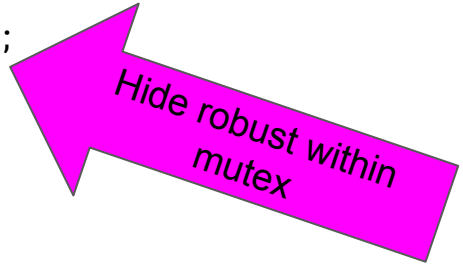
#define FUTEX_WAITERS         0x80000000
#define FUTEX_OWNER_DIED      0x40000000

void init_thread(mythread_t *thread)
{
     struct robust_list_head *rl = &thread->robust_list;
     memset(rl, 0, sizeof(*rl));
     rl->list.next = &rl->list;
     rl->list.offest = offsetof(struct my_mutex, lock) -
                       offsetof(struct my_mutex, list);
     set_robust_list(rb, sizeof(*rl));
}
```

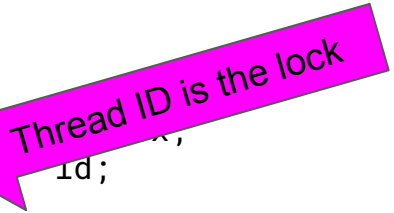


```c
int robust_lock(struct mutex *mutex)
{
	struct my_mutex *rf = (struct my_mutex *)mutex;
	int old2, old = 0, id = gettid(), new = id;
	for (;;) {
		old2 = old;
		old = cmpxchg(&rf->lock, old2, new);
		if (old == old2) {
			set_robust(rf);
			return 0;
		}
check:
		if (old & FUTEX_OWNER_DIED)
			return -1;
		switch (old) {
		case 0: new = id; continue;
		case FUTEX_WAITERS: new = old | id; continue;
		}
		old2 = old;
		old = cmpxchg(&rf->lock, old2, old2 | id);
		if (old != old2) goto check;
		futex(&rf->lock, FUTEX_WAIT, old, NULL);
		old = rf->lock & FUTEX_WAITERS;
		goto check;
	}
}
```

```c
int robust_lock(struct mutex *mutex)
{
	struct my_mutex *rf = (struct my_mutex *)mutex;
	int old2, old = 0, id = gettid(), new = id;
	for (;;) {
		old2 = old;
		old = cmpxchg(&rf->lock, old2, new);
		if (old == old2) {
			set_robust(rf);
			return 0;
		}
check:
		if (old & FUTEX_OWNER_DIED)
			return -1;
		switch (old) {
		case 0: new = id; continue;
		case FUTEX_WAITERS: new = old | id; continue;
		}
		old2 = old;
		old = cmpxchg(&rf->lock, old2, old2 | id);
		if (old != old2) goto check;
		futex(&rf->lock, FUTEX_WAIT, old, NULL);
		old = rf->lock & FUTEX_WAITERS;
		goto check;
	}
}
```



```
int robust_lock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mu                    x;
     int old2, old = 0, id = gettid(), n          id;
     for (;;) {
          old2 = old;
          old = cmpxchg(&rf->lock, old2, new);
          if (old == old2) {
               set_robust(rf);
               return 0;
          }
check:
          if (old & FUTEX_OWNER_DIED)
               return -1;
          switch (old) {
          case 0: new = id; continue;
          case FUTEX_WAITERS: new = old | id; continue;
          }
          old2 = old;
          old = cmpxchg(&rf->lock, old2, old2 | id);
          if (old != old2) goto check;
          futex(&rf->lock, FUTEX_WAIT, old, NULL);
          old = rf->lock & FUTEX_WAITERS;
          goto check;
     }
}
```



```
int robust_lock(struct mutex *mutex)
{
      struct my_mutex *rf = (struct my_mutex *)mutex;
      int old2, old = 0, id = gettid(), new = id;
      for (;;) {
            old2 = old;
            old = cmpxchg(&rf->l                  new);
            if (old == old2) {
                  set_robust(rf);
                  return 0;
            }
check:
            if (old & FUTEX_OWNER_DIED)
                  return -1;
            switch (old) {
            case 0: new = id; continue;
            case FUTEX_WAITERS: new = old | id; continue;
            }
            old2 = old;
            old = cmpxchg(&rf->lock, old2, old2 | id);
            if (old != old2) goto check;
            futex(&rf->lock, FUTEX_WAIT, old, NULL);
            old = rf->lock & FUTEX_WAITERS;
            goto check;
      }
}
```



```
int robust_lock(struct mutex *mutex)
{
      struct my_mutex *rf = (struct my_mutex *)mutex;
      int old2, old = 0, id = gettid(), new = id;
      for (;;) {
            old2 = old;
            old = cmpxchg(&rf->lock, old2
            if (old == old2) {
                  set_robust(rf);
                  return 0;
            }
check:
            if (old & FUTEX_OWNER_DIED)
                  return -1;
            switch (old) {
            case 0: new = id; continue;
            case FUTEX_WAITERS: new = old | id; continue;
            }
            old2 = old;
            old = cmpxchg(&rf->lock, old2, old2 | id);
            if (old != old2) goto check;
            futex(&rf->lock, FUTEX_WAIT, old, NULL);
            old = rf->lock & FUTEX_WAITERS;
            goto check;
      }
}
```



```
int robust_lock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     int old2, old = 0, id = gettid(), new = id;
     for (;;) {
          old2 = old;
          old = cmpxchg(&rf->lock, old2, new);
          if (old == old2) {
               set_robust(rf);
               return 0;
          }
check:
          if (old & FUTEX_OWNER_DIED)
               return -1;
          switch (old) {
          case 0: new = id; continue;
          case FUTEX_WAITERS: new = old | id; continue;
          }
          old2 = old;
          old = cmpxchg(&rf->lock, old2, old2 | id);
          if (old != old2) goto check;
          futex(&rf->lock, FUTEX_WAIT, old, NULL);
          old = rf->lock & FUTEX_WAITERS;
          goto check;
     }
}
```



```
int robust_lock(struct mutex *mutex)
{
      struct my_mutex *rf = (struct my_mutex *)mutex;
      int old2, old = 0, id = gettid(), new = id;
      for (;;) {
            old2 = old;
            old = cmpxchg(&rf->lock, old2, new);
            if (old == old2) {
                  set_robust(rf);
                  return 0;
            }
check:
            if (old & FUTEX_O
                  return -1;
            switch (old) {
            case 0: new = id; continue;
            case FUTEX_WAITERS: new = old | id; continue;
            }
            old2 = old;
            old = cmpxchg(&rf->lock, old2, old2 | id);
            if (old != old2) goto check;
            futex(&rf->lock, FUTEX_WAIT, old, NULL);
            old = rf->lock & FUTEX_WAITERS;
            goto check;
      }
}
```

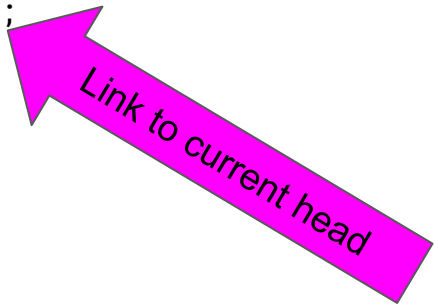


```
static __thread mythread_t thread;

void set_robust(struct my_mutex *rl)
{
     rl->list.next = thread.robust_list.next;
     thread.robust_list.next = &rl->list;
}
```

```
static __thread mythread_t thread;

void set_robust(struct my_mutex *rl)
{
     rl->list.next = thread.robust_list.next;
     thread.robust_list.next = &rl->list;
}
```



```
static __thread mythread_t thread;

void set_robust(struct my_mutex *rl)
{
     rl->list.next = thread.robust_list.next;
     thread.robust_list.next = &rl->list;
}
```



```
static __thread mythread_t thread;

void set_robust(struct my_mutex *rl)
{
     rl->list.next = thread.robust_list.next;
     thread.robust_list.next = &rl->list;
}
```

Now if we die, the kernel will know!

```
static __thread mythread_t thread;

void set_robust(struct my_mutex *rl)
{
     rl->list.next = thread.robust_list.next;
     thread.robust_list.next = &rl->list;
}
```



```c
struct robust_list {
     struct robust_list  *next;
};

struct robust_list_head {
     struct robust_list  list;
     long                offset;
     struct robust_list  *list_op_pending;
};

struct my_mutex {
     unsigned int        lock;
     struct robust_list  list;
};

#define FUTEX_WAITERS          0x80000000
#define FUTEX_OWNER_DIED       0x40000000

void init_thread(mythread_t *thread)
{
     struct robust_list *rl = &thread->robust_list;
     memset(rl, 0, sizeof(*rl));
     rl->list.next = &rl->list;
     rl->list.offest = offsetof(struct my_mutex, lock) -
                       offsetof(struct my_mutex, list);
     set_robust_list(rb, sizeof(*rl));
}
```



list_op_pending

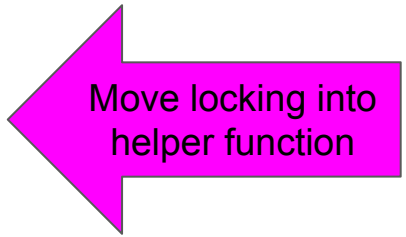
- Before locking assign the `list_op_pending` to the lock we are about to grab

list_op_pending

- Before locking assign the `list_op_pending` to the lock we are about to grab
- If the task crashes, the kernel will look at this lock
  - If it holds the TID of the task, it will treat that lock as owner died

```
static int try_robust_lock(struct my_mutex *rf, int *old, int new)
{
     int old2;

     old2 = old;
     *old = cmpxchg(&rf->lock, old2, new);
     if (*old == old2) {
          set_robust(rf);
          return 1;
     }
     return 0;
}
```

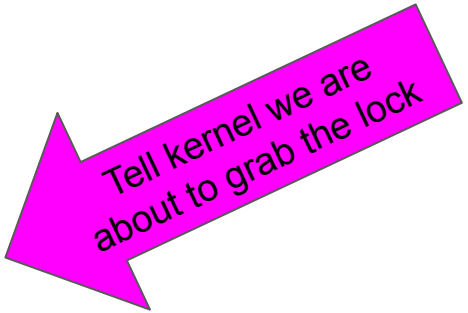


```c
static __thread mythread_t thread;

int robust_lock(struct my_mutex *rf, int *old, int id)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     int ret, id = gettid(), new = id;
     for (;;) {
          thread.robust_list.list_op_pending = &rf->list.next;
          ret = try_robust_lock(rf, &old, new);
          thread.robust_list.list_op_pending = NULL;
          if (ret)
               return 0;
check:
          if (old & FUTEX_OWNER_DIED)
               return -1;
          switch (old) {
          case 0: new = id; continue;
          case FUTEX_WAITERS: new = old | id; continue;
          }
          old2 = old;
          old = cmpxchg(&rf->lock, old2, old2 | id);
          if (old != old2) goto check;
          futex(&rf->lock, FUTEX_WAIT, old, NULL);
          old = rf->lock & FUTEX_WAITERS;
          goto check;
     }
}
```

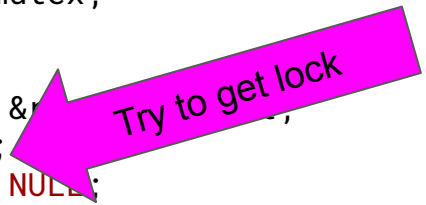
```c
static __thread mythread_t thread;

int robust_lock(struct my_mutex *rf, int *old, int id)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     int ret, id = gettid(), new = id;
     for (;;) {
          thread.robust_list.list_op_pending = &rf->list.next;
          ret = try_robust_lock(rf, &old, new);
          thread.robust_list.list_op_pending = NULL;
          if (ret)
               return 0;
check:
          if (old & FUTEX_OWNER_DIED)
               return -1;
          switch (old) {
          case 0: new = id; continue;
          case FUTEX_WAITERS: new = old | id; continue;
          }
          old2 = old;
          old = cmpxchg(&rf->lock, old2, old2 | id);
          if (old != old2) goto check;
          futex(&rf->lock, FUTEX_WAIT, old, NULL);
          old = rf->lock & FUTEX_WAITERS;
          goto check;
     }
}
```



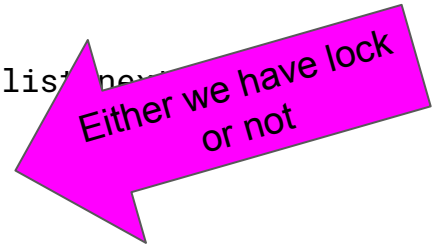
```c
static __thread mythread_t thread;

int robust_lock(struct my_mutex *rf, int *old, int id)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     int ret, id = gettid(), new = id;
     for (;;) {
          thread.robust_list.list_op_pending = &r
          ret = try_robust_lock(rf, &old, new);
          thread.robust_list.list_op_pending = NULL;
          if (ret)
               return 0;
check:
          if (old & FUTEX_OWNER_DIED)
               return -1;
          switch (old) {
          case 0: new = id; continue;
          case FUTEX_WAITERS: new = old | id; continue;
          }
          old2 = old;
          old = cmpxchg(&rf->lock, old2, old2 | id);
          if (old != old2) goto check;
          futex(&rf->lock, FUTEX_WAIT, old, NULL);
          old = rf->lock & FUTEX_WAITERS;
          goto check;
     }
}
```



```
static __thread mythread_t thread;

int robust_lock(struct my_mutex *rf, int *old, int id)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     int ret, id = gettid(), new = id;
     for (;;) {
          thread.robust_list.list_op_pending = &rf->list_next;
          ret = try_robust_lock(rf, &old, new);
          thread.robust_list.list_op_pending = NULL;
          if (ret)
               return 0;
check:
          if (old & FUTEX_OWNER_DIED)
               return -1;
          switch (old) {
          case 0: new = id; continue;
          case FUTEX_WAITERS: new = old | id; continue;
          }
          old2 = old;
          old = cmpxchg(&rf->lock, old2, old2 | id);
          if (old != old2) goto check;
          futex(&rf->lock, FUTEX_WAIT, old, NULL);
          old = rf->lock & FUTEX_WAITERS;
          goto check;
     }
}
```



```
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; (*next)->next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_list.next;
     old = cmpxchg(&rf->lock, id, 0);
     if (old == id)
          goto out;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
out:
     thread.robust_list.list_op_pending = NULL;
}
```

```
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

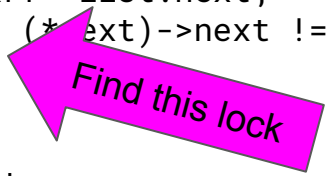
     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; (*next)->next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_list.next;
     old = cmpxchg(&rf->lock, id, 0);
     if (old == id)
          goto out;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
out:
     thread.robust_list.list_op_pending = NULL;
}
```



```c
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

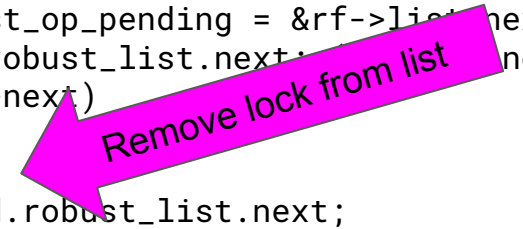
     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; (*next)->next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_list.next;
     old = cmpxchg(&rf->lock, id, 0);
     if (old == id)
          goto out;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
out:
     thread.robust_list.list_op_pending = NULL;
}
```



```
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; *next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_list.next;
     old = cmpxchg(&rf->lock, id, 0);
     if (old == id)
          goto out;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
out:
     thread.robust_list.list_op_pending = NULL;
}
```



```
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; (*next)->next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_list.
     old = cmpxchg(&rf->lock, id, 0);
     if (old == id)
          goto out;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
out:
     thread.robust_list.list_op_pending = NULL;
}
```



```c
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; (*next)->next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_list.next;
     old = cmpxchg(&rf->lock, id, 0);
     if (old == id)
          goto out;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
out:
     thread.robust_list.list_op_pending = NULL;
}
```



```
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; (*next)->next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_list.next;
     old = cmpxchg(&rf->lock, id, 0);
     if (old == id)
          goto out;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE, 1);
     if (!waiters)
          futex(&rf->lock, FUTEX_WAITERS, 0);
out:
     thread.robust_list.list_op_pending = NULL;
}
```

Race?

The Ugly

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock
- Some data has refcounts and locks to synchronize destruction

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock
- Some data has refcounts and locks to synchronize destruction
- When the lock is unlocked, the lock can be freed

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock
- Some data has refcounts and locks to synchronize destruction
- When the lock is unlocked, the lock can be freed
  - Task A unlocks and gets preempted before updating `list_op_pending`

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock
- Some data has refcounts and locks to synchronize destruction
- When the lock is unlocked, the lock can be freed
  - Task A unlocks and gets preempted before updating `list_op_pending`
  - Task B locks sets condition that the lock is not used anymore

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock
- Some data has refcounts and locks to synchronize destruction
- When the lock is unlocked, the lock can be freed
  - Task A unlocks and gets preempted before updating `list_op_pending`
  - Task B locks sets condition that the lock is not used anymore
  - Task B unlocks, frees the mutex, remaps the mutex

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock
- Some data has refcounts and locks to synchronize destruction
- When the lock is unlocked, the lock can be freed
  - Task A unlocks and gets preempted before updating `list_op_pending`
  - Task B locks sets condition that the lock is not used anymore
  - Task B unlocks, frees the mutex, remaps the mutex
  - Task B adds a value to the lock address that matches Task A's TID

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock
- Some data has refcounts and locks to synchronize destruction
- When the lock is unlocked, the lock can be freed
  - Task A unlocks and gets preempted before updating `list_op_pending`
  - Task B locks sets condition that the lock is not used anymore
  - Task B unlocks, frees the mutex, remaps the mutex
  - Task B adds a value to the lock address that matches Task A's TID
  - Task A triggers an exception and exits

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock
- Some data has refcounts and locks to synchronize destruction
- When the lock is unlocked, the lock can be freed
  - Task A unlocks and gets preempted before updating `list_op_pending`
  - Task B locks sets condition that the lock is not used anymore
  - Task B unlocks, frees the mutex, remaps the mutex
  - Task B adds a value to the lock address that matches Task A's TID
  - Task A triggers an exception and exits
  - The kernel sees Task A's list_op_pending and cleans up the mutex

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock
- Some data has refcounts and locks to synchronize destruction
- When the lock is unlocked, the lock can be freed
  - Task A unlocks and gets preempted before updating `list_op_pending`
  - Task B locks sets condition that the lock is not used anymore
  - Task B unlocks, frees the mutex, remaps the mutex
  - Task B adds a value to the lock address that matches Task A's TID
  - Task A triggers an exception and exits
  - The kernel sees Task A's list_op_pending and cleans up the mutex
  - The kernel adds the FUTEX_OWNER_DIED flag

The Robust Futex Unlock Bug

- Has to do with another task removing the mutex after unlock
- Some data has refcounts and locks to synchronize destruction
- When the lock is unlocked, the lock can be freed
  - Task A unlocks and gets preempted before updating `list_op_pending`
  - Task B locks sets condition that the lock is not used anymore
  - Task B unlocks, frees the mutex, remaps the mutex
  - Task B adds a value to the lock address that matches Task A's TID
  - Task A triggers an exception and exits
  - The kernel sees Task A's list_op_pending and cleans up the mutex
  - The kernel adds the FUTEX_OWNER_DIED flag
  - Task B's value is now corrupted!

The Robust Futex Unlock Bug

- First reported in 2012
  - https://sourceware.org/bugzilla/show_bug.cgi?id=14485

The Robust Futex Unlock Bug

- First reported in 2012
  - https://sourceware.org/bugzilla/show_bug.cgi?id=14485
- Partially fixed THIS YEAR!

The Robust Futex Unlock Bug

- First reported in 2012
  - https://sourceware.org/bugzilla/show_bug.cgi?id=14485
- Partially fixed THIS YEAR!
- Mathieu Desnoyers proposed a fix via VDSO
  - https://lore.kernel.org/all/20260311185409.1988269-1-mathieu.desnoyers@efficios.com/

The Robust Futex Unlock Bug

- First reported in 2012
  - https://sourceware.org/bugzilla/show_bug.cgi?id=14485
- Partially fixed THIS YEAR!
- Mathieu Desnoyers proposed a fix via VDSO
  - https://lore.kernel.org/all/20260311185409.1988269-1-mathieu.desnoyers@efficios.com/
- Thomas Gleixner extended Mathieu's idea
  - https://lore.kernel.org/all/20260602084648.462672743@kernel.org/

The Robust Futex Unlock Bug

- First reported in 2012
  - https://sourceware.org/bugzilla/show_bug.cgi?id=14485
- Partially fixed THIS YEAR!
- Mathieu Desnoyers proposed a fix via VDSO
  - https://lore.kernel.org/all/20260311185409.1988269-1-mathieu.desnoyers@efficios.com/
- Thomas Gleixner extended Mathieu's idea
  - https://lore.kernel.org/all/20260602084648.462672743@kernel.org/
  - Added in 7.2 (the latest Linux release)

The Robust Futex Unlock Bug

- First reported in 2012
  - https://sourceware.org/bugzilla/show_bug.cgi?id=14485
- Partially fixed THIS YEAR!
- Mathieu Desnoyers proposed a fix via VDSO
  - https://lore.kernel.org/all/20260311185409.1988269-1-mathieu.desnoyers@efficios.com/
- Thomas Gleixner extended Mathieu's idea
  - https://lore.kernel.org/all/20260602084648.462672743@kernel.org/
  - Added in 7.2 (the latest Linux release)
  - Introduced `FUTEX_ROBUST_UNLOCK` and `__vdso_futex_robust_try_unlock()`

futex_robust_try_unlock() (Pseudo Code)

```
int futex_robust_try_unlock(int *lock, int tid, void *pop)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;

     asm volatile (“
          xor            %ecx,%ecx      // Set ECX to 0
          lock cmpxchg   %ecx,(%rdi)    // Try the TID -> 0 transition
  .Lstart:
          jnz            .Lend
          movq           %rcx,(%rdx)    // Clear list_op_pending
  .Lend:
          “ : “+&a”(tid) : “D”(lock), “d”(pop)”);

     return tid;
}
```

futex_robust_try_unlock() (Pseudo Code)

```
int futex_robust_try_unlock(int *lock, int tid, void *pop)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;

     asm volatile (“
          xor            %ecx,%ecx      // Set ECX
          lock cmpxchg   %ecx,(%rdi)    // Try transition
  .Lstart:
          jnz            .Lend
          movq           %rcx,(%rdx)    // Clear list_op_pending
  .Lend:
          “ : “+&a”(tid) : “D”(lock), “d”(pop)”);

     return tid;
}
```

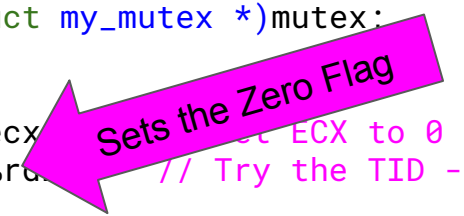


futex_robust_try_unlock() (Pseudo Code)

```
int futex_robust_try_unlock(int *lock, int tid, void *pop)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;

     asm volatile (“
          xor             %ecx,%ecx          // Set ECX to 0
          lock cmpxchg    %ecx,(%rd          // Try the TID -> 0 transition
  .Lstart:
          jnz             .Lend
          movq            %rcx,(%rdx)        // Clear list_op_pending
  .Lend:
          “ : “+&a”(tid) : “D”(lock), “d”(pop)”);

     return tid;
}
```



futex_robust_try_unlock() (Pseudo Code)

```
int futex_robust_try_unlock(int *lock, int tid, void *pop)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;

     asm volatile (“
          xor            %ecx,%ecx      // Set ECX to 0
          lock cmpxchg   %ecx,(%rdi)    // Try the TID -> 0 transition
  .Lstart:
          jnz            .Lend
          movq           %rcx,(%rdx)    // Clear list_op_pending
  .Lend:
          “ : “+&a”(tid) : “D”(lock), “d”(pop)”);

     return tid;
}
```

```
list_op_pending = regs->flags & X86_EFLAGS_ZF ? NULL : regs->dx;
```

futex_robust_try_unlock() (Pseudo Code)

```
int futex_robust_try_unlock(int *lock, int tid, void *pop)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;

     asm volatile (“
          xor            %ecx,%ecx      // Set ECX to 0
          lock cmpxchg   %ecx,(%rdi)    // Try the TID -> 0 transition
  .Lstart:
          jnz            .Lend
          movq           %rcx,(%rdx)    // Clear list_op_pending
  .Lend:
          “ : “+&a”(tid) : “D”(lock), “d”(pop)”);

     return tid
}
```

Kernel Code

```
list_op_pending = regs->flags & X86_EFLAGS_ZF ? NULL : regs->dx;
```

```
static __thread mythread_t thread;

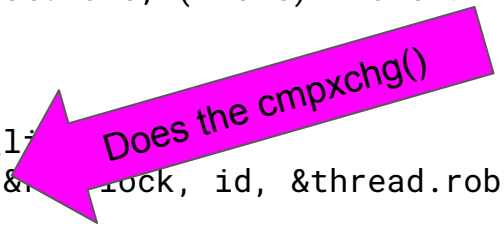
void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; (*next)->next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_list.next;
     old = futex_robust_try_unlock(&rf->lock, id, &thread.robust_list.list_op_pending);
     if (old == id)
          return;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE|FUTEX_ROBUST_UNLOCK, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
}
```

```
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; (*next)->next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_li
     old = futex_robust_try_unlock(&     lock, id, &thread.robust_list.list_op_pending);
     if (old == id)
          return;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE|FUTEX_ROBUST_UNLOCK, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
}
```



```
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

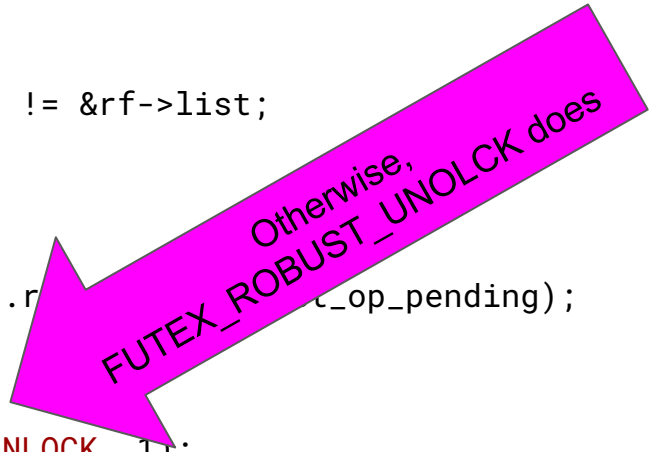
     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread-    bust_list.next; (*next)->next != &rf->list;
          next = &(*next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_
     old = futex_robust_try_unlock(&rf->lo      d, &thread.robust_list.list_op_pending);
     if (old == id)
          return;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE|FUTEX_ROBUST_UNLOCK, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
}
```

On success, clears list_op_pending

```
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

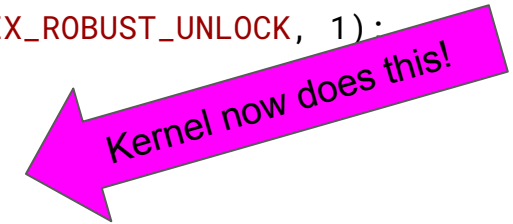
     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; (*next)->next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_list.next;
     old = futex_robust_try_unlock(&rf->lock, id, &thread.r                t_op_pending);
     if (old == id)
          return;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE|FUTEX_ROBUST_UNLOCK, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
}
```



```c
static __thread mythread_t thread;

void robust_unlock(struct mutex *mutex)
{
     struct my_mutex *rf = (struct my_mutex *)mutex;
     struct robust_list **next;
     int old, waiters, id = gettid();

     thread.robust_list.list_op_pending = &rf->list.next;
     for (next = &thread->robust_list.next; (*next)->next != &rf->list;
          next = &(*next)->next)
          ;
     *next = (*next)->next;
     rl->list.next = thread.robust_list.next;
     old = futex_robust_try_unlock(&rf->lock, id, &thread.robust_list.list_op_pending);
     if (old == id)
          return;
     *lock = FUTEX_WAITERS;
     waiters = futex(&rf->lock, FUTEX_WAKE|FUTEX_ROBUST_UNLOCK, 1);
     if (!waiters)
          cmpxchg(&rf->lock, FUTEX_WAITERS, 0);
// out:
//   thread.robust_list.list_op_pending = NULL;
}
```



Other ugliness

- PI futex
  - Does priority inheritance
  - Great for RT tasks

Other ugliness

- PI futex
  - Does priority inheritance
  - Great for RT tasks
  - Sucks for non-RT tasks (a fair lock)
    - A topic for another day

Other ugliness

- PI futex
  - Does priority inheritance
  - Great for RT tasks
  - Sucks for non-RT tasks (a fair lock)
    - A topic for another day
  - Work to fix this (PI Next Generation)

Other ugliness

- PI futex
  - Does priority inheritance
  - Great for RT tasks
  - Sucks for non-RT tasks (a fair lock)
    - A topic for another day
  - Work to fix this (PI Next Generation)
    - PING!

Other ugliness

- PI futex
  - Does priority inheritance
  - Great for RT tasks
  - Sucks for non-RT tasks (a fair lock)
    - A topic for another day
  - Work to fix this (PI Next Generation)
    - PING!
- Condition variables
  - This can be a talk on to itself

Questions?



Questions?

